Sommaire

1. Pourquoi une base perd des données
2. Le journal d'écriture anticipée
3. fsync, le mensonge des caches
4. Points de reprise et rejeu
5. Réplication : le journal voyage
6. Ce qu'il faut retenir

1. Pourquoi une base perd des données

Une base de données promet qu'une transaction validée survit à une panne. Cette promesse s'appelle la durabilité, le D de ACID. Elle est plus difficile à tenir qu'il n'y paraît, car entre l'instruction `COMMIT` et la surface magnétique ou les cellules flash, une donnée traverse au moins trois mémoires volatiles : le tampon de la base, le cache de pages du noyau et le cache interne du disque.

Une coupure de courant efface les trois. Une base qui répond « validé » alors que la donnée n'a quitté que le premier tampon ment à son client. Le problème n'est donc pas d'écrire, mais de savoir quand une écriture est réellement arrivée.

Écrire directement les pages de données modifiées à chaque validation serait correct mais lent : une transaction qui touche dix tables provoque dix écritures aléatoires, dispersées sur le disque. Les disques, même SSD, préfèrent de loin les écritures séquentielles.

2. Le journal d'écriture anticipée

L'idée centrale, connue sous le nom de WAL (*write-ahead log*), tient en une règle : avant de modifier une page de données sur le disque, on écrit dans un journal la description de la modification, et on attend que ce journal soit durable.

Le journal est un fichier où l'on ne fait qu'ajouter à la fin. Chaque enregistrement porte un numéro croissant, le LSN (*log sequence number*), qui ordonne toutes les modifications de la base. Valider une transaction revient à écrire son enregistrement de validation dans le journal puis à le rendre durable : une seule écriture séquentielle, quel que soit le nombre de tables touchées.

Les pages de données, elles, peuvent rester modifiées en mémoire et rejoindre le disque plus tard, en lot. Si la machine tombe avant, le journal contient de quoi les reconstruire. On échange ainsi des écritures aléatoires synchrones contre des écritures séquentielles synchrones et des écritures aléatoires différées.

Plusieurs transactions qui valident au même instant peuvent partager une même écriture du journal : c'est la validation groupée (*group commit*), qui multiplie le débit quand la latence du disque domine.

3. fsync, le mensonge des caches

Sous Linux, `write()` copie les octets dans le cache de pages du noyau et rend la main aussitôt. Rien n'est encore sur le disque. Il faut appeler `fsync()` sur le fichier pour demander au noyau de pousser ces pages et d'ordonner au disque de vider son propre cache.

Deux pièges reviennent souvent. D'abord, créer ou renommer un fichier modifie le répertoire qui le contient : sans `fsync()` sur le répertoire, le fichier peut disparaître après une panne alors que son contenu était durable. Ensuite, quand `fsync()` échoue, le noyau peut marquer les pages comme propres et les oublier ; rappeler `fsync()` réussira alors sans avoir rien écrit. En 2018, PostgreSQL a dû changer sa politique : après un échec de `fsync()`, il s'arrête et rejoue le journal au lieu de réessayer.

Enfin, certains disques grand public annoncent qu'un vidage de cache est terminé alors qu'il ne l'est pas. Seuls des disques avec protection contre les coupures, ou un cache désactivé, rendent la promesse vraie jusqu'au bout.

4. Points de reprise et rejeu

Si le journal grandit sans fin, la reprise après panne devient interminable. Le point de reprise (*checkpoint*) borne ce travail : la base écrit sur le disque toutes les pages modifiées jusqu'à un certain LSN, puis note ce LSN. Après une panne, il suffit de rejouer le journal à partir de ce point.

Le rejeu (*redo*) applique chaque enregistrement dont le LSN est supérieur à celui inscrit sur la page concernée. Comme chaque page garde le LSN de sa dernière modification, rejouer deux fois le même enregistrement ne change rien : l'opération est idempotente, ce qui rend la reprise elle-même robuste aux pannes.

Les segments de journal antérieurs au dernier point de reprise peuvent être supprimés, ou archivés pour permettre une restauration à un instant donné (*point-in-time recovery*) en rejouant le journal par-dessus une sauvegarde.

5. Réplication : le journal voyage

Puisque le journal décrit toutes les modifications dans l'ordre, l'envoyer à une autre machine suffit à en faire une copie. C'est le principe de la réplication physique en flux (*streaming replication*) : le réplica rejoue en continu le journal reçu du primaire.

La question devient alors : quand répondre « validé » au client ? En mode asynchrone, dès que le journal est durable sur le primaire ; une panne du primaire peut perdre les dernières transactions. En mode synchrone, après qu'au moins un réplica a confirmé avoir reçu, voire écrit, l'enregistrement ; aucune perte, au prix d'un aller-retour réseau à chaque validation.

Les systèmes de consensus comme Raft généralisent cette idée : un journal répliqué, où une entrée est validée quand une majorité de nœuds l'a rendue durable.

6. Ce qu'il faut retenir

- La durabilité se joue entre `COMMIT` et le média physique, à travers plusieurs caches.
- Le WAL transforme des écritures aléatoires synchrones en une écriture séquentielle.
- `fsync()` est la seule frontière fiable, y compris sur le répertoire.
- Le point de reprise borne le temps de reprise ; le LSN rend le rejeu idempotent.
- Répliquer, c'est expédier le journal ; synchrone ou non, c'est choisir quand dire oui.